

TP2 : Algorithme Random Forest (Forêt Aléatoire)

Matière : Intelligence Artificielle

Responsable : Pr. S. Chabaa

1. Objectif du TP

Ce TP a pour objectif de présenter et d'implémenter l'algorithme **Random Forest**, une méthode d'apprentissage supervisé basée sur un ensemble d'arbres de décision.

L'algorithme Random Forest est utilisé pour résoudre :

- un problème de **régression** visant à estimer une grandeur physique continue ;
- un problème de **classification** permettant d'identifier l'état de fonctionnement d'un système électrique.

Ce TP s'appuie sur des données réelles issues d'un système de génie électrique, afin de mettre en évidence l'intérêt des méthodes d'ensemble dans l'analyse des systèmes industriels.

2. Présentation du système et des données

Le système considéré est un moteur électrique opérant sous des régimes de fonctionnement variables, ce qui induit des variations significatives de ses caractéristiques thermiques et électriques. Certaines grandeurs d'état sont directement accessibles par mesure, tandis que d'autres, en particulier la température interne du moteur, ne peuvent pas être mesurées directement en exploitation et doivent être estimées à partir de variables mesurables.

Les données utilisées sont issues de mesures expérimentales enregistrées dans un fichier CSV et comprennent :

- le courant absorbé RMS I_{RMS} ,
- la température ambiante T_{amb} ,
- la température interne du moteur T_{mot} ,
- le taux de distorsion harmonique du courant (THD),
- l'état de fonctionnement du moteur (normal / anormal).

Ces variables permettent d'aborder des problématiques de régression et de classification à l'aide de l'algorithme Random Forest.

Partie 1 : Régression (estimation de T_{mot})

Étape 1 — Chargement des bibliothèques et des données

Dans cette première étape, les bibliothèques Python nécessaires sont importées, puis le fichier de données est chargé afin d'extraire les variables utiles.

Code Python

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.preprocessing import MinMaxScaler
6 from sklearn.model_selection import train_test_split
7
8 data = pd.read_csv("moteur_data.csv")
9
10 Irms = data["Irms"].values
11 Tamb = data["Tamb"].values
12 Tmot = data["Tmot"].values
13 THD = data["THD"].values
14 Etat = data["Etat"].values
```

Explication ligne par ligne

- RandomForestRegressor : implémente Random Forest pour la régression.
- Les autres bibliothèques permettent le calcul numérique, la gestion des données et la visualisation.
- Les colonnes du fichier CSV sont extraites sous forme de tableaux NumPy.

Étape 2 — Analyse exploratoire des données

Avant l'apprentissage, il est nécessaire de visualiser les données afin d'observer la relation entre le courant absorbé et la température interne.

Code Python

```
1 plt.figure(figsize=(6,4))
2 plt.scatter(Irms, Tmot)
3 plt.xlabel("Courant_RMS_(A)")
4 plt.ylabel("Temp rature_interne_( C )")
5 plt.grid(True)
```

```
6 plt.show()
```

Étape 3 — Préparation et normalisation des données

Même si Random Forest n'est pas sensible à l'échelle des variables, une normalisation est réalisée afin d'assurer une cohérence méthodologique avec les autres algorithmes étudiés.

Code Python

```
1 X = np.column_stack((Irms, Tamb))
2
3 scaler = MinMaxScaler()
4 Xn = scaler.fit_transform(X)
```

Étape 4 — Application de Random Forest en régression

Le modèle Random Forest est entraîné sur les données d'apprentissage afin d'estimer la température interne du moteur.

Code Python

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     Xn, Tmot, test_size=0.25, random_state=1)
3
4 rf = RandomForestRegressor(
5     n_estimators=100,
6     random_state=1
7 )
8
9 rf.fit(X_train, y_train)
10 y_pred = rf.predict(X_test)
```

Étape 5 — Analyse des résultats

Les valeurs estimées sont comparées aux valeurs réelles. La droite $y = x$ représente le cas idéal.

Code Python

```
1 plt.figure(figsize=(6,4))
2 plt.scatter(y_test, y_pred)
3 plt.plot([Tmot.min(), Tmot.max()],
4          [Tmot.min(), Tmot.max()], 'r--')
5 plt.xlabel("Temp rature_r elle_( C )")
6 plt.ylabel("Temp rature_estim e_( C )")
7 plt.grid(True)
8 plt.show()
```

Partie 2 : Classification (normal / anormal)

Étape 1 — Application de Random Forest en classification

Dans cette partie, Random Forest est utilisé pour classier l'état du moteur à partir du courant RMS et du THD.

Code Python

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 Xc = np.column_stack((Irms, THD))
4
5 scaler = MinMaxScaler()
6 Xc_n = scaler.fit_transform(Xc)
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     Xc_n, Etat, test_size=0.25, random_state=2)
10
11 rf = RandomForestClassifier(
12     n_estimators=100,
13     random_state=2
14 )
15
16 rf.fit(X_train, y_train)
17 y_pred = rf.predict(X_test)
```

Étape 2 — Visualisation des classes

Code Python

```

1 plt.figure(figsize=(6,4))
2 plt.scatter(Irms[Etat==0], THD[Etat==0], label="Normal")
3 plt.scatter(Irms[Etat==1], THD[Etat==1], label="Anormal")
4 plt.xlabel("Courant_RMS_(A)")
5 plt.ylabel("THD_(%)")
6 plt.legend()
7 plt.grid(True)
8 plt.show()

```

Étape 3 — Évaluation du modèle de classification

Après l'apprentissage du modèle Random Forest pour la classification, il est indispensable d'évaluer ses performances afin de vérifier sa capacité à identifier correctement l'état de fonctionnement du moteur (normal ou anormal).

Deux indicateurs complémentaires sont utilisés :

- l'**accuracy**, qui mesure le pourcentage global de classifications correctes ;
- la **matrice de confusion**, qui permet d'analyser en détail les erreurs de classification.

Code Python

```

1 from sklearn.metrics import accuracy_score, confusion_matrix,
   ConfusionMatrixDisplay
2
3 accuracy = accuracy_score(y_test, y_pred)
4 print("Accuracy_du_mod le_Random_Forest:", accuracy)
5
6 cm = confusion_matrix(y_test, y_pred)
7 ConfusionMatrixDisplay(confusion_matrix=cm).plot()
8 plt.title("Matrice_de_confusion_Random_Forest")
9 plt.show()

```

Explication ligne par ligne

- `from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay` : importe les fonctions nécessaires à l'évaluation des performances du modèle de classification.
- `accuracy_score(y_test, y_pred)` : calcule le taux de prédictions correctes, en comparant les classes réelles (`y_test`) aux classes prédites (`y_pred`).
- `print(...)` : affiche la valeur de l'accuracy, exprimée entre 0 et 1 (ou en pourcentage).
- `confusion_matrix(y_test, y_pred)` : génère la matrice de confusion, qui indique le nombre de bonnes et mauvaises classifications pour chaque classe.

- `ConfusionMatrixDisplay(...).plot()` : affiche graphiquement la matrice de confusion pour une meilleure interprétation.
- `plt.title(...)` : ajoute un titre à la figure pour préciser l'algorithme utilisé.
- `plt.show()` : affiche la figure à l'écran.

Étape 4 — Évaluation du modèle de régression

Dans le cas de la régression, l'objectif est d'évaluer la précision de l'estimation de la température interne du moteur. L'indicateur retenu est la **RMSE (Root Mean Squared Error)**, qui mesure l'erreur moyenne entre les valeurs réelles et les valeurs prédites.

Une valeur faible de la RMSE indique une bonne capacité du modèle à estimer la grandeur physique étudiée.

Code Python

```
1 from sklearn.metrics import mean_squared_error
2
3 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
4 print("RMSE du mod le Random Forest:", rmse)
```

Explication ligne par ligne

- `from sklearn.metrics import mean_squared_error` : importe la fonction permettant de calculer l'erreur quadratique moyenne.
- `mean_squared_error(y_test, y_pred)` : calcule la moyenne des carrés des écarts entre les valeurs réelles et les valeurs estimées par le modèle.
- `np.sqrt(...)` : applique la racine carrée afin d'obtenir la RMSE, exprimée dans la même unité que la variable cible (ici, le degré Celsius).
- `print(...)` : affiche la valeur de la RMSE pour interprétation.

Exercice 2

Dans cet exercice, nous nous intéressons à l'utilisation de l'algorithme *Random Forest* dans le contexte des bâtiments intelligents. À partir de données issues de capteurs environnementaux (température, humidité, luminosité, concentration en CO₂), l'objectif est d'aborder deux problématiques complémentaires du machine learning supervisé.

La première consiste à résoudre un problème de **classification** visant à identifier l'état d'occupation d'un bureau (occupé ou non occupé). La seconde porte sur un problème de **régression**, dont l'objectif est d'estimer la température intérieure du bâtiment à partir des mêmes variables explicatives.

L'algorithme Random Forest, basé sur un ensemble d'arbres de décision, est particulièrement adapté à ce type de données hétérogènes et non linéaires, et permet d'obtenir des modèles robustes et performants.

Travail demandé

1. Charger le jeu de données fourni.
2. Analyser et visualiser les variables d'entrée et de sortie.
3. Implémenter un modèle **Random Forest** pour estimer l'état d'occupation du bureau.
4. Évaluer les performances du modèle de classification à l'aide de l'accuracy et de la matrice de confusion.
5. Implémenter un modèle **Random Forest** pour prédire la température intérieure du bâtiment.
6. Évaluer les performances du modèle de régression à l'aide de la RMSE.